MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatic

REPORT ON LABORATORY WORK No. 3

Course: Cryptography and Security

Done by: student, group FAF-243
Andrei Elciscev

Checked by:
Zaica Maia

Chișinău, 2026

Contents

1 Introduction **2**

2 Objectives **2**

3 Theoretical Background **2**

3.1 Classical Substitution Ciphers 2

3.2 The Vigenère Cipher 3

3.3 The Playfair Cipher 3

3.4 Character Set Used in the Implementation 4

3.5 Construction of the Playfair Matrix 4

3.6 Preparation of Digraphs 5

3.7 Encryption Rules 5

3.7.1 Rule 1: Same Row 5

3.7.2 Rule 2: Same Column 5

3.7.3 Rule 3: Rectangle 6

3.8 Decryption Rules 6

4 Implementation **6**

4.1 Development Environment 6

4.2 Character Normalization and Validation 7

4.3 Matrix Construction 7

4.4 Digraph Preparation 7

4.5 Encryption and Decryption 7

4.6 Console Interface 8

5 Testing and Results **8**

5.1 Test 1: Playfair Encryption 8

5.2 Test 2: Playfair Decryption 9

5.3 Test 3: Invalid Operation 10

5.4 Test 4: Invalid Keyword 10

5.5 Test 5: Invalid Plaintext Character 11

5.6 Test 6: Odd-Length Ciphertext 11

5.7 Testing Summary 12

6 Conclusions **12**

1 Introduction

Cryptography studies methods of protecting information by transforming readable messages into forms that are difficult for unauthorized individuals to understand. Encryption and decryption are fundamental operations in cryptographic systems.

Classical cryptography includes several substitution techniques. Monoalphabetic substitution replaces each plaintext character with a corresponding ciphertext character using a fixed substitution. Such ciphers are vulnerable to frequency analysis because the relationship between plaintext and ciphertext characters remains constant.

Polyalphabetic ciphers use multiple substitution alphabets, while digraphic substitution ciphers encrypt pairs of characters rather than individual characters.

This laboratory work studies the principles of the Vigenère and Playfair ciphers. The practical implementation is a C# console application that performs Playfair encryption and decryption using a 5-by-6 matrix and a configured set of Romanian characters.

2 Objectives

The objectives of this laboratory work are:

- To study the principles of classical substitution ciphers.
- To understand the operation of the Vigenère cipher.
- To study the encryption and decryption rules of the Playfair cipher.
- To construct a 5-by-6 Playfair matrix from a keyword.
- To adapt the matrix to the character set configured in the C# implementation.
- To implement Playfair encryption and decryption.
- To validate user input and handle invalid data.
- To test the application using valid and invalid inputs.

3 Theoretical Background

3.1 Classical Substitution Ciphers

A substitution cipher replaces plaintext characters with other characters according to a defined rule.

In a monoalphabetic substitution cipher, each plaintext character always corresponds to the same ciphertext character. In a polyalphabetic cipher, the substitution can change depending on the character position and key.

The Playfair cipher differs from ordinary single-character substitution because it encrypts pairs of characters, known as digraphs. The characters' positions in a matrix determine the substitution applied to each pair.

3.2 The Vigenère Cipher

The Vigenère cipher is a classical polyalphabetic substitution cipher. It uses a repeating keyword to determine the shift applied to each plaintext character.

For the traditional English alphabet, each letter is represented by a number from 0 to 25, where $A = 0$ and $Z = 25$.

The encryption formula is:

$$C_i = (P_i + K_i) \bmod 26$$

The decryption formula is:

$$P_i = (C_i - K_i + 26) \bmod 26$$

where:

- P_i is the numerical value of the plaintext character;
- K_i is the numerical value of the corresponding key character;
- C_i is the numerical value of the ciphertext character;
- i is the character position.

The Vigenère cipher is discussed here as theoretical background. The provided application source code implements the Playfair cipher rather than the Vigenère cipher.

3.3 The Playfair Cipher

The Playfair cipher is a digraphic substitution cipher associated with Charles Wheatstone and Lord Playfair. It encrypts pairs of characters instead of processing every character independently.

The cipher uses a matrix constructed from a keyword. Repeated characters in the keyword are removed, and the remaining characters from the configured alphabet are appended in their defined order.

The traditional English Playfair cipher commonly uses a 5-by-5 matrix, combining I and J to fit the 26-letter English alphabet into 25 cells.

The implementation in this laboratory uses a 5-by-6 matrix containing 30 configured characters.

3.4 Character Set Used in the Implementation

The program defines the following character set:

A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

The configured array contains 30 distinct characters. It includes Romanian-specific characters such as Ă, Â, Î, Ș, and Ț, but excludes W. Therefore, it is a configured character set based on the Romanian alphabet rather than the complete 31-letter Romanian alphabet.

The matrix dimensions are defined in the source code as:

```
public static readonly int Rows = 5;
public static readonly int Cols = 6;
```

Consequently, the matrix has exactly 30 cells. Every cell is occupied by one character from the configured set. Digits are not inserted into the matrix.

The program also normalizes Romanian cedilla variants to their comma-below equivalents. For example, Ş is normalized to Ș, and Ţ is normalized to Ț.

3.5 Construction of the Playfair Matrix

The `BuildMatrix` method constructs the matrix according to the following procedure:

1. Normalize the keyword and convert it to uppercase.
2. Select distinct valid characters from the keyword, preserving their original order.
3. Place those characters into the matrix.
4. Append the remaining characters from the configured alphabet in the order defined by the program.
5. Skip any character that has already been inserted.

A set of placed characters prevents duplicates from appearing in the matrix. Since the configured alphabet contains 30 characters and the matrix has 30 cells, the matrix is filled without additional symbols.

The same keyword must be used for encryption and decryption because it determines the matrix arrangement.

3.6 Preparation of Digraphs

Before encryption, the input is normalized and converted to uppercase. All whitespace characters are removed. The remaining characters are validated against the configured character set.

The plaintext is then divided into pairs according to these rules:

- If two consecutive characters are different, they form a digraph.
- If two consecutive characters are identical, a filler character is inserted after the first character. The second original character is processed again.
- If one character remains at the end, a filler character is appended to complete the final pair.

The implementation uses `X` as the default filler character. If the character being processed is `X`, the program uses `Q` instead.

For example, when two identical characters occur consecutively, the first character is paired with a filler. Processing then continues from the second original character.

These rules ensure that the prepared plaintext consists of complete digraphs without identical characters inside a pair.

3.7 Encryption Rules

Let the positions of two characters in a digraph be:

$$A = (r_1, c_1), \qquad B = (r_2, c_2)$$

The encryption procedure applies one of three rules.

3.7.1 Rule 1: Same Row

If both characters occupy the same row, each character is replaced by the character immediately to its right. If a character is in the last column, the operation wraps around to the first column.

3.7.2 Rule 2: Same Column

If both characters occupy the same column, each character is replaced by the character immediately below it. If a character is in the last row, the operation wraps around to the first row.

3.7.3 Rule 3: Rectangle

If the characters occupy different rows and different columns, they form opposite corners of a rectangle. Each character is replaced by the character in its own row and the other character's column.

The resulting pair is appended to the ciphertext.

3.8 Decryption Rules

Decryption reverses the encryption transformations:

- **Same row:** move one position to the left, wrapping around to the last column when necessary.
- **Same column:** move one position upward, wrapping around to the last row when necessary.
- **Rectangle:** exchange the column positions using the same rectangle rule.

The ciphertext must contain an even number of valid characters. The implementation checks the length and throws an `ArgumentException` if it is odd.

The decrypted output may contain filler characters inserted during encryption. The program does not automatically remove them because a filler character may also have appeared in the original plaintext.

4 Implementation

4.1 Development Environment

The application was implemented in C# as a console program. It is divided into two principal components:

- `PlayfairCipher`: provides character normalization, matrix construction, input validation, digraph preparation, encryption, decryption, and matrix display functionality.
- `Program`: manages user interaction, checks the selected operation and keyword, invokes the cipher methods, and displays results or error messages.

4.2 Character Normalization and Validation

The `NormalizeInput` method normalizes Unicode input and replaces the Romanian cedilla variants Ş and Ţ with Ș and Ț.

The `IsAllowedLetter` method checks whether a character belongs to the configured character set after case conversion and normalization.

The console interface checks the keyword before starting the selected operation. It requires at least seven non-whitespace characters and rejects characters that do not belong to the configured set.

During plaintext and ciphertext processing, whitespace is removed. Other unsupported characters, including punctuation and digits, cause an error.

4.3 Matrix Construction

The `BuildMatrix` method creates a 5-by-6 character matrix. It first inserts distinct valid characters from the keyword, then adds the remaining characters from `RomanianAlphabet`.

The `FindPosition` method searches the matrix for a specified character and returns its row and column coordinates. These coordinates are used to determine which Playfair rule applies to each digraph.

4.4 Digraph Preparation

The `PrepareDigraphs` method normalizes the plaintext, converts it to uppercase, removes whitespace, and validates the remaining characters.

It then processes the input from left to right. Identical consecutive characters are separated by a filler, and a remaining final character is paired with a filler. If the input is empty or contains an unsupported character, the method returns an error rather than continuing with encryption.

4.5 Encryption and Decryption

The `Encrypt` method constructs the matrix, prepares the plaintext digraphs, finds the positions of each pair, and applies the corresponding encryption rule.

The `Decrypt` method constructs the same matrix, normalizes and validates the ciphertext, and checks that its character count is even. If the length is odd, it throws an exception with the message that the ciphertext length must be even.

Both methods return the resulting text as a string.

4.6 Console Interface

The console interface displays the permitted character set and allows the user to select one of three options:

- `E` – encrypt plaintext;
- `D` – decrypt ciphertext;
- `X` – exit the application.

An unsupported operation produces an error message, after which the application continues its main loop.

For encryption, the program requests plaintext and displays the ciphertext and the matrix used. For decryption, it requests ciphertext and displays the raw decrypted digraphs and the matrix.

The application informs the user that spaces must be reinserted manually and that filler characters may need to be removed manually.

5 Testing and Results

The application was tested using valid operations and different invalid-input scenarios. The following screenshots document the corresponding console interactions.

5.1 Test 1: Playfair Encryption

This test demonstrates encryption with a valid keyword and plaintext. The application prepares the plaintext digraphs, encrypts each pair according to the Playfair rules, and displays the resulting ciphertext and matrix.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): bunĂ ziua

Ciphertext: CSOHȚLSB

Matrix used:
A B C U S T
E H Ă Â D F
G I Î J K L
M N O P Q R
Ș Ț V X Y Z
```

Figure 1: Playfair encryption with a valid keyword and plaintext

5.2 Test 2: Playfair Decryption

This test demonstrates decryption using the keyword corresponding to the matrix used for encryption. The program validates the ciphertext, decrypts each pair, and displays the resulting text and matrix.

The decrypted result may contain filler characters introduced during plaintext preparation.

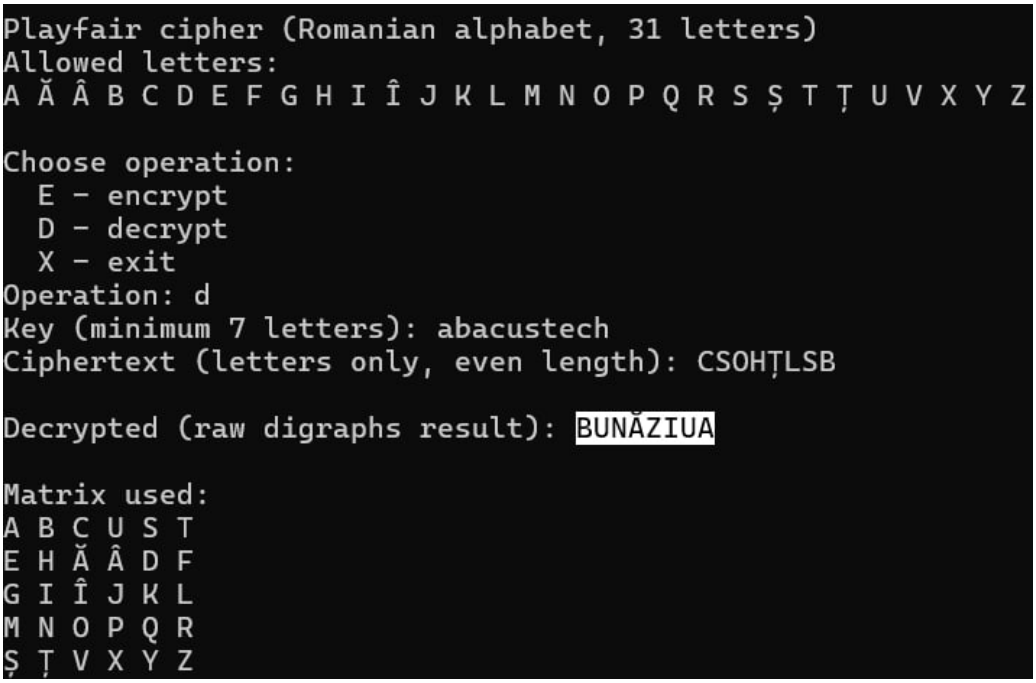


Figure 2: Playfair decryption

5.3 Test 3: Invalid Operation

This test checks the program's response when the user enters an operation other than `E`, `D`, or `X`. The application displays an error message and returns to the operation-selection prompt.

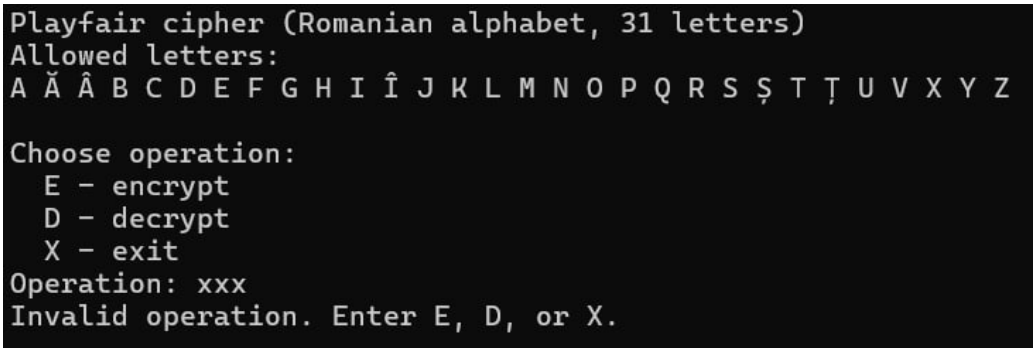


Figure 3: Validation of an invalid operation

5.4 Test 4: Invalid Keyword

This test checks the minimum keyword length requirement. The program rejects a keyword containing fewer than seven non-whitespace characters. It also rejects unsupported characters in the keyword.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): nope
Key too short. Minimum length is 7 letters.
```

Figure 4: Validation of an invalid keyword

5.5 Test 5: Invalid Plaintext Character

This test demonstrates validation during plaintext preparation. If the plaintext contains an unsupported non-whitespace character, the program reports the character and displays the allowed character set.

Whitespace is removed before validation, but punctuation and digits are not silently removed.

```
Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): v0lat1le
Error: Character '0' is not allowed. Allowed letters are: A Ă Â
```

Figure 5: Validation of an unsupported plaintext character

5.6 Test 6: Odd-Length Ciphertext

The Playfair cipher processes ciphertext in pairs of characters. Therefore, the implementation requires an even number of ciphertext characters.

The `Decrypt` method checks the ciphertext length before processing its digraphs. If the number of characters is odd, the method throws an `ArgumentException` with the message `Ciphertext length must be even.`

The application catches the exception and displays the error message. It does not append a filler character or automatically modify the ciphertext.

```
Operation: e
Key (minimum 7 letters): abacustech
Plaintext (letters only): impar

Ciphertext: GNMUPZ

Matrix used:
A B C U S T
E H Ă Â D F
G I Î J K L
M N O P Q R
Ș Ț V X Y Z

Note: Reinsert spaces manually if needed.

Playfair cipher (Romanian alphabet, 31 letters)
Allowed letters:
A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V X Y Z

Choose operation:
  E - encrypt
  D - decrypt
  X - exit
Operation: d
Key (minimum 7 letters): abacustech
Ciphertext (letters only, even length): GNMUPZ

Decrypted (raw digraphs result): IMPARX
```

Figure 6: Rejection of ciphertext with an odd number of characters

5.7 Testing Summary

The tests cover encryption, decryption, invalid operations, keyword validation, unsupported plaintext characters, and odd-length ciphertext.

The implementation uses the keyword to construct the matrix and applies the Playfair rules to each character pair. It also validates the character set and ciphertext length before processing the input.

The original spaces and formatting are not preserved. Successful decryption recovers the prepared plaintext, which may differ from the original plaintext because filler characters can be inserted during encryption.

6 Conclusions

During this laboratory work, the principles of classical substitution ciphers were studied, with particular attention to the Vigenère and Playfair ciphers.

A Playfair encryption and decryption application was implemented in C#. It uses a

5-by-6 matrix containing the 30 characters defined in the program's configured character set. The character set includes Romanian-specific letters and excludes W.

The application constructs the matrix from a keyword, removes duplicate characters, prepares plaintext digraphs, and encrypts or decrypts pairs according to the Playfair rules. Input validation handles unsupported characters, invalid operations, keywords that do not meet the minimum length requirement, and ciphertexts with an odd number of characters.

The tests demonstrate the program's behavior for valid encryption and decryption as well as several invalid-input conditions. The implementation also illustrates the importance of character normalization, matrix construction, and input validation in cryptographic software.

Although classical ciphers are useful for understanding cryptographic principles, they are not suitable for protecting sensitive modern communications because their security is limited by known cryptanalytic techniques.